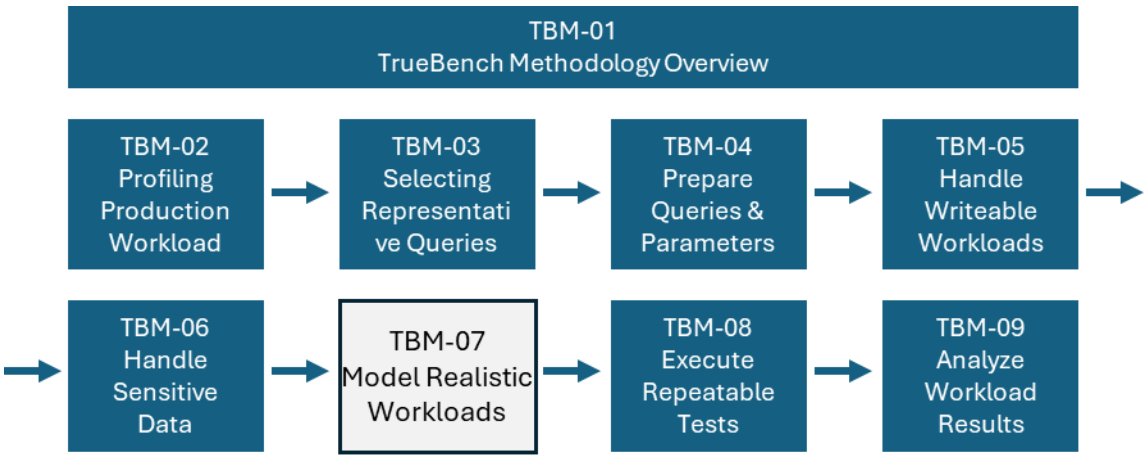
Model Realistic Workloads

Ensuring test reveals system drift.

Author: Douglas H Ebel

Data Warehouse Architect
Business Value and Performance Assessment

Creator of TdBench and TrueBench



Introduction

Most systems are not idle and then suddenly become active. They operate as continuous flows of overlapping activity.

Is this:

A good simulation of this?

A common mistake in testing is to simulate concurrent activity with query scripts that start at the same time and execute queries in a fixed sequence. This resembles a starting line.

Real systems behave differently. Queries arrive at different times, run for different durations, and overlap in uneven patterns. This resembles a crowd.

In earlier steps, we focused on selecting representative queries. That work ensured we are testing the right components of the system.

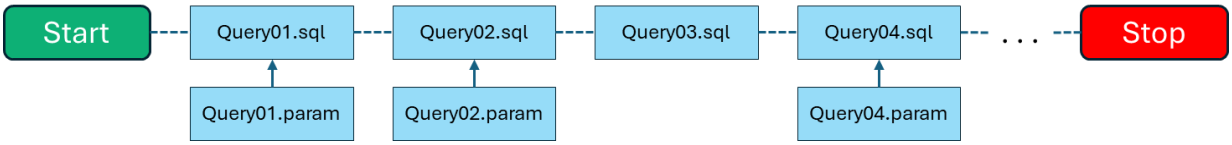
Now, we must execute those queries in a way that preserves their behavior together. The objective remains the same: to detect drift that impacts both individual queries and the system as a whole.

There are two modes of validation:

- **Serial test,** which executes each query independently
- **Workload test,** which simulates the arrival patterns of the production system

Serial Execution (Query-Level Validation)

Each query is executed independently, one at a time. This identifies DBMS and library changes that impact a single query.



Purpose:

- Validate that all prepared SQL and parameter-generation logic execute correctly
- Establish baseline behavior for each query
- Measure resource consumption characteristics

This serial mode isolates the effect of change. If a query regresses here, the cause is intrinsic to that query.

Example: Initial Serial Test:

```
define serial first test
queue q1 scripts/*.sql
param q1 param/*.param
worker q1 mydb
run
```

Description:

- The test name is “serial” and given a title of “first test”
- All queries in the scripts directory are executed
- If a matching .param file exists, the first parameter row is applied
- A single worker session is used (default)
- Each query is executed once

Basic Test Assessment:

- TrueBench automatically records each test in its SQLite TestTracking table. Prior tests can be viewed with the LIST command.
- For each test, the execution of every query is recorded in the TestResults table. You can use the SELECT command to identify the QueryName and ErrorText for a RunID that had a non-zero ReturnCode.
- You can use the NOTE command to record anything about a RunID that may help you later in separating good tests from those with issues.

Design Considerations for Workload Tests

Workload tests identify changes that impact multiple users. Examples:

- Platform changes impacting available resources
- "Noisy neighbors" using shared cloud infrastructure are impacting performance
- Changes to workload management that favor different users.
- Optimizer changes that cause queries to become skewed and use a disproportionate amount of shared resources, impacting other queries

To be valid, the tests must realistically simulate the query execution environment. One common mistake is misunderstanding the level of concurrency. Specifically:

#Registered Users ≠ #Users logged on ≠ Queries in flight

A good rule of thumb to determine concurrency is:

- 1/10th of the registered users = # users logged on
- 1/10th of the users logged = #queries in flight

That means as a maximum:

10,000 registered users = 1,000 logged on users = 100 queries in flight

Bad Examples:

- One insurance company ran two workload tests, initiating 500 identical queries at the same time and 1,000 queries at the same time. Each test was completed in less than one minute. What is the likelihood of this occurring in the real world? Based on this flawed test, they selected a DBMS that later failed under their actual workload.
- One vendor provided a query driver intended to simulate the number of logged-on users by introducing random "think time" between 10 and 30 seconds across multiple threads. First, this does not account for the full range of real user behavior—composing queries, reviewing results, answering the phone, interacting with coworkers, or stepping away. Second, it merely enables claims about "users supported" without any measurable connection to actual system behavior.

TrueBench Realistic Workload Modeling:

TrueBench methodology advocates executing queries deterministically, without random "think time." This ensures that comparisons across test executions are valid and repeatable. Rather than focusing on "number of users supported," this approach emphasizes measurable workload throughput, such as Queries Per Hour (QPH).

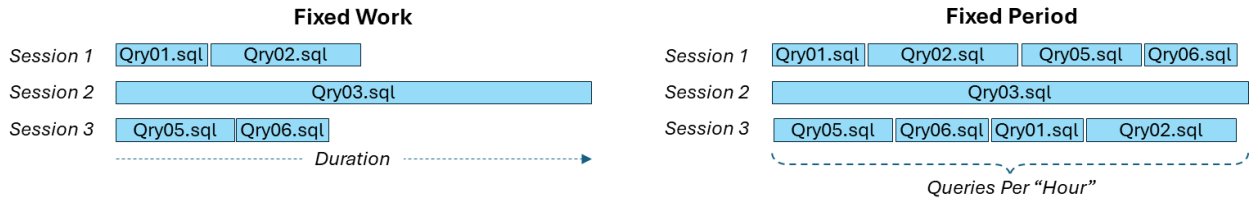
To support this modeling, analysis of actual concurrency provides useful input. Appendix A demonstrates a query-based approach for determining the number of queries in flight and waiting to execute, by type, throughout the day. This data can be exported for further analysis to examine overall concurrency patterns or to drill into specific workload characteristics such as query type, source, or average duration. This provides a data-driven basis for selecting realistic concurrency levels rather than relying on assumptions about user activity

With TrueBench, the information system workload is modeled by assigning queries to queues based on their origin (sales, marketing, etc.) and/or their characteristics (heavy, medium, light). Concurrency is determined by the number of workers (a.k.a. execution threads) assigned to each work queue. While any number of queues can be defined, most often 3-5 queues are sufficient. Some queues can be defined with specific timing to simulate events that stress the platform's workload management or scaling capabilities.

The test script may also include the PACE command, which regulates the rate of query arrivals per second, minute, or hour, or specifies the percentage of total query executions assigned to one or more queues. Without PACE, test results can become highly skewed. For example, short-running general ledger queries may be executed tens of thousands of times during a test period, while longer-running business queries may be executed only hundreds of times. It is not realistic for 30,000 executions of a single query to occur within a 30-minute test window.

Fixed Work or Fixed Period Tests?

Fixed work tests are easy to construct without a query driver, but fail to represent realistic workloads. Fixed period tests simulate a consistent workload for the duration of the test.



There are key flaws with fixed-work tests:

1. Light queries may complete quickly, causing the total test duration to be driven by the longest-running queries. The resulting metric primarily reflects the behavior of those longest queries, with limited insight into overall workload behavior.

2. To compensate, identical scripts containing the same set of queries may be executed in each session. This causes the same queries to be executed at approximately the same time across sessions.

3. Shuffling queries within each concurrent session addresses the synchronization issue in #2, but introduces a different problem: all queries are still executed the same number of times, which does not reflect realistic execution ratios.

Modeling Realistic Workload

A fixed-period test maintains a consistent level of activity throughout the test duration, but additional design is required to simulate a realistic workload.

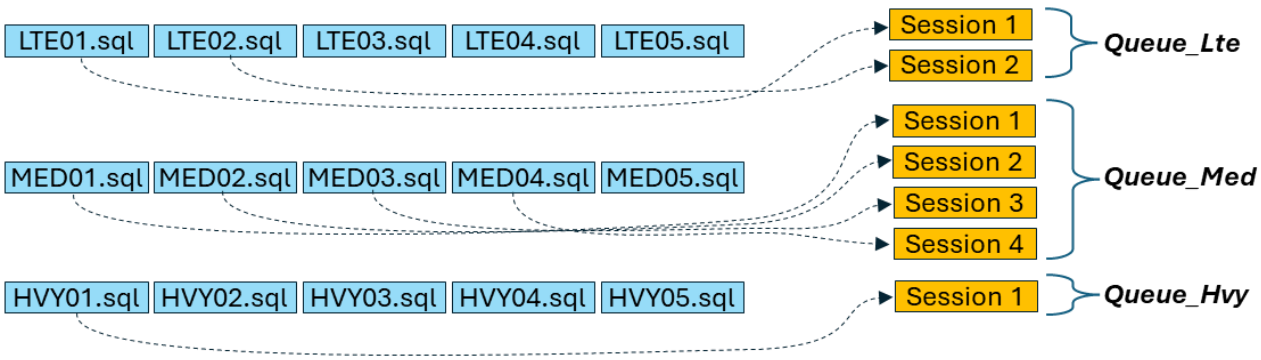
Workloads should model the relative execution frequencies of different query types. For example, call center queries may occur much more frequently than marketing queries and typically execute more quickly.

TrueBench supports multiple queues, allowing work units (SQL statements) to be separated based on:

- origin (sales, marketing, call center, etc.)
- execution characteristics (heavy, medium, light)

The production workload can then be modeled by assigning a different number of sessions (workers in TrueBench) to each queue.

Example:



Controlling Concurrency and Execution Mix

The number of sessions determines how many queries can execute concurrently.

However, without additional control, queues may still process unrealistic volumes of work. The PACE command provides two mechanisms to address this:

- regulate the rate of query arrival (per second, minute, or hour)
- control the percentage of total executions assigned to each queue

This allows the workload to reflect both:

- concurrency levels
- relative execution frequency across query types

In one test for an airline system, the workload required:

- 7,300 trip retrievals per hour
- 20,000 ticket retrievals per hour

The PACE command enabled these rates to be modeled precisely within the workload test.

Assembling the Validation Test Suite

The objective of TBM-07 is to develop a set of tests that detect drift at both the query and system levels. This is achieved through a combination of serial and workload tests.

Serial Test

The serial test is executed first. Purpose:

- Validate correctness of all queries and parameter sets
- Establish baseline behavior for each query
- Detect regressions affecting individual queries

Workload Tests

Workload tests execute queries under controlled concurrency and arrival patterns to simulate realistic system activity. These tests detect:

- contention for shared resources
- workload management effects
- changes in query interaction
- shifts in overall system behavior

Scaling Workload Intensity

Workload tests are typically executed at increasing levels of concurrency to observe how system behavior changes as demand increases. Examples:

- 10, 20, 40, 80 total sessions

- or 4, 8, 16 sessions in smaller or resource-constrained environments

The number of sessions is distributed across queues based on the modeled workload.

Interpreting Results

The objective is not simply to identify the system's maximum throughput. Instead, these tests are used to observe:

- how response times change as concurrency increases
- whether specific queries begin to dominate resource usage
- whether contention appears earlier than expected
- whether workload balance shifts between queues

Beyond Response Time

While response time is a primary metric, deeper analysis may be required to understand observed behavior. TrueBench integrates with host DBMS query logging to enable detailed analysis of resource usage and execution characteristics.

TrueBench can automatically maintain a TestTracking table within the host DBMS, capturing test identifiers, timestamps, and execution statistics. This simplifies the extraction of corresponding query log data for each test run.

In addition, TrueBench automatically inserts query identification markers into each SQL statement. These identifiers allow each execution recorded in the host DBMS logs to be precisely correlated to the specific query defined in the test.

Together, these capabilities enable analysis of:

- CPU and I/O consumption
- query skew and resource imbalance
- execution plan changes
- concurrency patterns (queries in flight over time)

This level of analysis allows validation to move beyond detecting change to explaining its cause.

Summary

A complete validation suite consists of:

- a serial test to establish query-level baselines
- a set of workload tests at increasing intensity levels

- consistent workload modeling using queues, workers, and pacing

Together, these tests provide a repeatable method for detecting and explaining drift in system behavior.

Appendix A: Measuring Queries in Flight from Host DBMS Query Logs

One useful way to validate a modeled workload is to compare it with observed production concurrency patterns.

A practical method is to calculate the number of queries that are:

- waiting to execute
- actively running

at regular time intervals throughout the day.

The following example query uses host DBMS query logging to produce a one-minute time series of:

- queries waiting to run
- queries in flight
- optional groupings by business area and statement type

This can help answer questions such as:

- When does concurrency peak?
- Is the workload dominated by SELECT activity, DML, or data loading?
- Are there periods where many queries are queued before first step execution?
- Does the production system show a steady flow of work or short bursts of activity?

Approach

The query uses three derived tables:

1. **SampleDate**- Defines the date to be analyzed.
2. **ql** - Extracts query log rows for that date and classifies them into useful categories such as:
- business group
- statement type
3. **tick** - Generates one row for each minute of the day.

The main query joins the generated minute ticks to the narrowed query log rows and counts whether, at each point in time, a query is:

- waiting to execute (`ParseDelayCnt`)
- actively running (`InFlightCnt`)

It also calculates elapsed waiting time and runtime values that can be used for additional analysis.

Notes

- The `SampleDate` value must be changed before execution.
- The grouping logic in the `ql` derived table should remain simple.
- A small number of categories works well.
- Avoid highly granular grouping columns such as raw `AppID` values, or the result set may become unnecessarily large.
- The example below references `pdcrinfo.DbqlogTbl_Hst`.
- Depending on the environment, it may be adapted to use `dbc.qrylog` or another query log source.

Sample SQL

```
-- This query creates a clock tick every 1 minute and looks to see what is parsing,
waiting, or running.
-- MANDITORY: The date needs to be changed in SampleDate.
--
-- This query can be adjusted if you have a simple categorization
-- of your user groups. Without any addition, it will produce no more than 1449 rows of
output.
-- If you define a grouping with a case statement resulting in 4 user groups, then you'll
get about 6,000 rows.
-- Don't use something like appid which has dozens of values or you may get 10's of
thousands of lines
-- ------------------------------------------------------------
With SampleDate as (select cast('2021-02-15' as date) dt), -- (select date - 1 dt),
  ql as (
       select
           StartTime,
           FirstStepTime,
           FirstRespTime,
           case
             when username like '%xxxxx1%' then 'xxxxxxx1'
             when username like '%xxxxx2%' then 'xxxxxxx2'
             -- etc
             else 'Other'
           end BusGrp,
           case
             when StatementType = 'Select' then 'Select'
             when StatementType = 'Collect Statistics' then 'Stats'
             when StatementType in ('Delete', 'Insert', 'Merge Into', 'Update', 'Create
Table') then 'DML'
```

```
             when StatementType in ('Begin Loading','Begin Mload','Begin
Transaction','Check Workload',
                                    'CheckPoint Loading','End Edit','End Loading','End
Transaction',
                                    'Execute Mload','Mload','Release Mload') then 'Load'
             else 'Other'
            end StmtType
       from pdcrinfo.DbqlogTbl_Hst, SampleDate
       where logdate = SampleDate.dt
       -- from dbc.qrylog, SampleDate where cast(StartTime as date) = SampleDate.dt
  ),
  tick as (
       select top 1439
              (rank() over(order by calendar_date)) * interval '1' minute
              + cast(SampleDate.dt as timestamp(0)) as tock
       from sys_calendar.calendar, SampleDate
       order by calendar_date
  )
select
   tick.tock,
   BusGrp,
   StmtType,
   Sum(case when tick.tock between StartTime and FirstStepTime  then 1 else 0 end)
ParseDelayCnt,
   Sum(case when tick.tock between FirstStepTime and FirstRespTime then 1 else 0 end)
InFlightCnt,
   Sum(case when tick.tock between FirstStepTime and FirstRespTime then
        ((FirstStepTime - StartTime HOUR(4)) (FLOAT)) * 3600. +
        ((FirstStepTime - ((FirstStepTime - StartTime HOUR(4))) - StartTime SECOND(4))
(FLOAT))
       else 0 end) AgeSecs,
   Sum(
        ((FirstStepTime - StartTime HOUR(4)) (FLOAT)) * 3600. +
        ((FirstStepTime - ((FirstStepTime - StartTime HOUR(4))) - StartTime SECOND(4))
(FLOAT))
      ) RunSecs
from tick
join ql
  on tick.tock between StartTime and FirstRespTime
group by 1,2,3
order by 1,2,3
;
```

Interpreting the Output

The result can be graphed in several ways.

A stacked area chart by statement type shows how the total number of active queries varies over time and which classes of work dominate each period.

A second chart can combine:

- queries in flight
- queries waiting to execute

This can help reveal periods where the platform is not simply busy, but is accumulating work faster than it can begin execution.

These patterns are useful when designing workload tests because they help identify:

- realistic concurrency levels
- dominant workload classes
- whether bursts or sustained periods of activity should be modeled

Sample Graphs